

LAB 4 – VIBE CODING

Math Flashcards Repair Lab

NAME: Khushi Gupta

SRN: PES1UG24AM912

Repository: 61_math_flashcards

Original Repository: SETAPESU26/61_math_flashcards

Student Repository: Khushi-tech4/61_math_flashcards

Technology: Python, Pygame

1. Objective

The objective of this laboratory exercise was to use an AI/LLM assistant as a debugging and pair-programming partner to repair a broken Pygame-based Math Flashcards application and implement the additional features specified in the repository README.

The assigned repository contained one deliberate bug and three additional features:

1. Fix the arithmetic calculation bug.
2. Implement a per-question countdown timer.
3. Implement consecutive-correct streak multipliers.
4. Add integer division with clean integer quotients.

The development process involved cloning the assigned repository, running the original application, recording the application before modification, using an LLM to analyze and implement the required changes, testing the application, and finally preparing the modified project for submission.

2. Repository Setup

User Action

The assigned repository was forked to my personal GitHub account.

The personal fork was:

https://github.com/Khushi-tech4/61_math_flashcards

The repository was then cloned locally using Git.

Command Used

```
cd "C:\Users\khush\OneDrive\Desktop\SE_LAB"
```

```
git clone https://github.com/Khushi-tech4/61_math_flashcards.git
```

Result

The repository was successfully cloned:

Cloning into '61_math_flashcards'...

remote: Enumerating objects: 14, done.

remote: Counting objects: 100% (14/14), done.

remote: Compressing objects: 100% (14/14), done.

remote: Total 14 (delta 2), reused 2 (delta 0), pack-reused 0

Receiving objects: 100% (14/14), 7.58 KiB | 1.08 MiB/s, done.

Resolving deltas: 100% (2/2), done.

The project directory was then opened:

```
cd 61_math_flashcards
```

3. Installing the Required Dependency

The project requires Pygame.

Initially, Pygame was installed using:

```
pip install pygame
```

Although the installation completed successfully, running the project using python main.py resulted in:

```
ModuleNotFoundError: No module named 'pygame'
```

Analysis

The error indicated that the pip command and the python command were using different Python environments.

The Python installation used by the project was identified as:

```
C:\Users\khush\AppData\Local\Programs\Python\Python311\python.exe
```

Therefore, Pygame was installed explicitly into that Python environment.

Command Used

```
"C:\Users\khush\AppData\Local\Programs\Python\Python311\python.exe" -m pip  
install pygame
```

The installation reported:

Requirement already satisfied: pygame

The installation was then verified using:

```
"C:\Users\khush\AppData\Local\Programs\Python\Python311\python.exe" -c "import  
pygame; print(pygame.version.ver)"
```

Result

The installed version was successfully detected:

pygame 2.6.1

SDL 2.28.4

Python 3.11.9

4. Running the Original Application

The original game was executed using:

```
"C:\Users\khush\AppData\Local\Programs\Python\Python311\python.exe" main.py
```

The application launched successfully without a Python import error.

The original gameplay was observed before making the required modifications.

A **10-second BEFORE video** was recorded as required by the laboratory instructions.

5. README Analysis

The repository README was reviewed to identify the required tasks.

The project is an interactive mental arithmetic flashcard game developed using Pygame.

The README identified four tasks.

Task 1 – Arithmetic Calculation Bug

The game incorrectly treats the operands as text instead of evaluating the mathematical operation.

For example:

5 + 7

should produce:

12

but the faulty implementation can concatenate the values and produce:

57

The calculation logic therefore needs to evaluate the active operator correctly.

Task 2 – Per-Question Timer

The game currently allows unlimited time for answering a question.

A countdown timer and visible timer bar must be added.

When the timer reaches zero, the question should automatically be registered as a missed attempt.

Task 3 – Consecutive Correct Streak Multiplier

The original scoring system provides a flat reward.

A streak-based multiplier must be introduced so that consecutive correct answers provide increasing rewards.

The streak must reset after an incorrect answer or timeout.

Task 4 – Integer Division

Division must be added to the available operators.

Generated division questions must always produce whole-number answers with no remainder.

6. Initial Git Status

After running and testing the application, the working tree was checked using:

git status

The repository reported:

On branch main

Your branch is up to date with 'origin/main'.

Changes not staged for commit:

modified: game/game_engine.py

modified: game/text_box.py

Untracked files:

game/__pycache__/

The __pycache__ directory is generated by Python and should not be included in the Git repository.

7. AI-Assisted Development Approach

The LLM was used as a debugging and pair-programming assistant rather than blindly replacing the complete project.

The intended workflow was:

Identify problem

↓

Ask LLM for analysis

↓

Review suggested solution

↓

Apply changes

↓

Run and test application

↓

Fix issues if required

↓

Commit the completed task

↓

Move to next task

Each feature was intended to be implemented independently so that each task could have a separate Git commit.

8. Planned Task-wise Git Commit Structure

The final Git history is intended to contain separate commits for the four required tasks:

Task 1: Fix arithmetic calculation

Task 2: Add per-question timer

Task 3: Add streak score multiplier

Task 4: Add integer division operator

This preserves a clear development history and demonstrates the individual completion of each task.

9. Testing Strategy

After implementing each task, the application was tested before moving to the next task.

Task 1 Testing

Test addition, subtraction, and multiplication using known answers.

Example:

$$5 + 7 = 12$$

$$10 - 4 = 6$$

$$6 * 3 = 18$$

Task 2 Testing

Verify that:

- The timer starts with each new question.
- The timer bar decreases.
- The question changes when time expires.
- A timeout is recorded as a missed attempt.

Task 3 Testing

Verify that:

- Correct answers increase the streak.
- The multiplier increases for consecutive correct answers.

- A wrong answer resets the streak.
- A timeout resets the streak.

Task 4 Testing

Verify that:

$$12 / 3 = 4$$

$$24 / 6 = 4$$

$$72 / 8 = 9$$

are possible, while questions producing fractional results are not generated.

10. Video Evidence

Two gameplay recordings were prepared according to the assignment requirements.

BEFORE Video

A 10-second recording was made before completing the required modifications.

The recording demonstrates the original state of the application.

AFTER Video

A second 10-second recording was made after completing the required modifications.

The recording demonstrates the repaired game and the implemented features.

12. Conclusion

The Math Flashcards project was successfully cloned from the personal fork and configured locally.

The Python/Pygame environment issue was identified and resolved by installing Pygame into the correct Python 3.11 environment.

The original application was successfully launched and the required BEFORE and AFTER gameplay recordings were prepared.

The remaining development workflow consists of verifying the four required tasks, maintaining separate Git commits for each task, testing the final application, and uploading the required Lab-4 deliverables to the personal laboratory repository.